

Mail-Server-Factory

Gestiona tu servidor de correo como un jefe — descríbelo en JSON y desplégalo donde quieras.

Resumen

Mail Server Factory es una herramienta de aprovisionamiento de servidores de correo automatizada y lista para producción. El usuario escribe una configuración sencilla en JSON; la Fábrica la interpreta y realiza todas las instalaciones e inicializaciones en el sistema operativo de destino, desplegando una pila de correo basada en Docker y con componentes débilmente acoplados, compatible con 12 tipos de conexión.

Descripción breve

Una herramienta Kotlin/Shell que convierte una descripción en JSON en un servidor de correo completamente instalado y dockerizado. Admite 12 tipos de conexión (SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt y más), un marco de seguridad completo, 25 distribuciones de Linux y se entrega con 439 pruebas superadas.

Descripción detallada

Poner en marcha un servidor de correo real y seguro es uno de los ritos de paso clásicos en la administración de sistemas, y también uno de los más fiablemente miserables. Postfix, Dovecot, certificados TLS, registros DNS, reglas de firewall y peculiaridades específicas de cada distribución deben alinearse a la perfección, y un solo error en una directiva puede significar correos rechazados en silencio o un relay abierto. Mail Server Factory toma todo ese conocimiento experto, difícil de dominar y propenso a errores, y lo plasma en software. En lugar de configurar manualmente cada componente en un sistema operativo desconocido, el usuario final describe el resultado deseado en un documento JSON sencillo; la Fábrica lee ese JSON y ejecuta los pasos exactos de instalación e inicialización

requeridos en el sistema operativo de destino, levantando una pila de correo que funciona sobre Docker, con todos sus componentes débilmente acoplados —un diseño que mantiene la pila escalable horizontalmente y permite actualizar o reemplazar cualquier componente de forma aislada—. Además, está deliberadamente diseñado para ser agnóstico en cuanto a la infraestructura: sus 12 tipos de conexión permiten que la misma herramienta y el mismo JSON apunten a una máquina local, un host remoto mediante SSH, un entorno Docker o Kubernetes, instancias en la nube a través de AWS SSM / Consola Serie de Azure / Inicio de sesión en GCP, o máquinas virtuales mediante Libvirt —la misma descripción declarativa, aprovisionada dondequiera que la dirijas—. Es compatible con 25 distribuciones de Linux de familias occidentales (Ubuntu, Debian, CentOS, Fedora, AlmaLinux, Rocky, openSUSE), rusas (ALT, Astra, ROSA) y chinas (openEuler, openKylin, Deepin), con instalación desatendida mediante preseed/kickstart/cloud-init/autoyast y automatización de máquinas virtuales basada en QEMU para pruebas. Las funciones empresariales son amplias: cifrado AES-256-GCM, políticas de contraseñas y claves SSH obligatorias, configuración automática de firewall para puertos de correo (25/587/465/993/995), TLS/SSL con validación de certificados e HSTS, registro de auditoría y RBAC. Entre las características operativas destacan el ajuste de la JVM (G1GC), caché Caffeine, agrupación de conexiones, métricas compatibles con Prometheus, registro estructurado, recarga en caliente de configuraciones y gestión de secretos. El proyecto cuenta con 439 pruebas superadas al 100% y supera la puerta de calidad SonarQube. Es el proyecto estrella de la organización Server-Factory.

Por qué lo creamos

Configurar un servidor de correo seguro y listo para producción es notoriamente propenso a errores y específico de cada sistema operativo. Mail Server Factory encapsula ese conocimiento en un modelo declarativo JSON junto con un motor de ejecución, de modo que una pila de correo correcta, segura y dockerizada pueda reproducirse en cualquier objetivo compatible sin necesidad de seguir pasos manuales uno a uno.

Por qué es un cambio radical

Transforma el aprovisionamiento de servidores de correo de un proceso especializado, que puede llevar días y requiere precisión absoluta, en un simple acto de escribir configuraciones. Además, hace que esa configuración sea portable entre 12 tipos de conexión y 25 distribuciones Linux, y viene endurecida con estándares de seguridad empresariales desde el primer momento. El resultado es reproducible y *verificable*: el mismo JSON genera

siempre la misma pila segura, y los 439 tests superados y la puerta limpia de SonarQube del proyecto garantizan que el motor que realiza el trabajo rinde cuentas en lugar de basarse en la reputación.

Qué tiene de innovador

- JSON declarativo → instalación/inicialización interpretada en el sistema operativo de destino.
- 12 tipos de conexión (local, SSH, Docker, Kubernetes, AWS SSM, Azure, GCP, Libvirt y más) gestionados con una sola herramienta.
- Soporte para 25 distribuciones con instalación desatendida (preseed/kickstart/cloud-init/autoyast) y automatización QEMU.
- Pila dockerizada de componentes débilmente acoplados para escalado y actualizaciones independientes.

Desafíos y soluciones

- **Heterogeneidad de sistemas operativos/distribuciones:** resuelto con recetas específicas para cada distribución, configuraciones de instalación desatendida y pruebas QEMU entre distintas distribuciones.
- **Alcanzar múltiples objetivos de despliegue:** resuelto con 12 tipos de conexión conectables bajo un motor de instalación común.
- **Seguridad por defecto:** resuelto con AES-256-GCM, políticas de claves/contraseñas obligatorias, reglas de firewall automatizadas y TLS/HSTS.
- **Confianza en la corrección:** resuelto con una suite de 439 tests (100 % superados) y una puerta limpia de SonarQube.

Tecnologías empleadas (por qué y cómo)

- **Kotlin** — el motor Factory y la lógica de pasos de instalación (179K bytes; Kotlin 2.0.21).
- **Shell** — scripts de aprovisionamiento, gestores de ISO/QEMU y automatización de sistemas operativos (predominante en bytes).
- **Docker** — el entorno de ejecución de la pila de correo desplegada, con componentes débilmente acoplados.
- **QEMU** — automatización de máquinas virtuales para instalación y pruebas entre distribuciones.
- **JSON** — el formato de configuración declarativa orientado al usuario.

- **Gradle 8.14.3 / Java 17** — cadena de herramientas de compilación.
- **Caffeine** — caché multirregión; **JVM con ajuste G1GC** para rendimiento.
- **Métricas compatibles con Prometheus** — monitorización; preparado para Grafana/ELK.
- **Sieve** — reglas de filtrado de correo (huella reducida en las estadísticas del lenguaje).

Nota: GitHub marca el repositorio como una bifurcación dentro de la organización Server-Factory. Es anterior a la línea de productos AI; se presenta como un buque insignia maduro de DevOps/aprovisionamiento, no como una utilidad de AI.